📘 NYC Taxi Trip Lakehouse Project

End-to-End Data Engineering using Databricks Medallion Architecture

---

1. Project Overview

1.1 Objective

The objective of this project is to design and implement an **enterprise-level Lakehouse Data Platform** using the **NYC Taxi Trip Dataset** on Databricks.

The project demonstrates:

- End-to-End Data Pipeline
- Medallion Architecture (Bronze → Silver → Gold)
- Fact & Dimension Modeling
- Unity Catalog Governance
- Volumes Usage
- Multiple Join Strategies
- Slowly Changing Dimensions (SCD)

1.2 Official Dataset

NYC Taxi Trip Dataset (Public Dataset)

https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

This dataset is widely used in:

- Databricks official demonstrations
- Azure Data Engineering interviews
- Enterprise lakehouse training environments

---

2. Dataset Description

2.1 Dataset Characteristics

Property	Description
Domain	Transportation Analytics

Property	Description
Data Volume	Millions of records per month
File Format	CSV / Parquet
Data Nature	Real production data
Update Frequency	Monthly

2.2 Dataset Files

Yellow Taxi Trips

Primary dataset containing detailed trip information.

Green Taxi Trips

Trips occurring mostly in outer borough areas.

FHV Trips

For-Hire Vehicle trip records.

2.3 Important Columns

Column	Description
VendorID	Taxi company identifier
pickup_datetime	Trip start time
dropoff_datetime	Trip end time
PULocationID	Pickup location
DOLocationID	Drop location
passenger_count	Number of passengers
trip_distance	Distance travelled
fare_amount	Base fare
total_amount	Total paid amount

---

3. Why This Dataset is Suitable

Requirement	Supported	Explanation
End-to-End Pipeline	Yes	Raw → Clean → Analytics
Medallion Architecture	Yes	Natural data layering
Fact & Dimension Modeling	Yes	Trips form fact table
Unity Catalog Volumes	Yes	Raw file storage
Join Operations	Yes	Lookup tables available
SCD Implementation	Yes	Location history tracking
Streaming Simulation	Yes	Monthly ingestion

---

4. Lakehouse Architecture

4.1 Architecture Flow

Cloud Storage (ADLS Gen2)

↓

Unity Catalog (Governance)

↓

Volumes (Raw Files)

↓

Bronze Tables

↓

Silver Tables

↓

Gold Tables (Star Schema)

↓

Analytics / BI Layer

---

5. Storage and Volume Design

5.1 Volume Structure

```
/Volumes/main/raw/nyc_taxi/
    ├── yellow/
    ├── green/
    ├── fhv/
    ├── lookup/
    └── schema/
```

5.2 Purpose of Volumes

Volumes are used to store:

- Raw ingestion files
- Incremental uploads
- Schema definitions
- Reference datasets

Example access:

```
spark.read.csv("/Volumes/main/raw/nyc_taxi/yellow/yellow_tripdata_2023.csv")
```

---

6. Medallion Architecture

6.1 Bronze Layer (Raw Layer)

Objective

Store raw data exactly as received.

Characteristics

- No transformation
- Append-only ingestion
- Schema preserved

Tables

Table	Description
bronze.raw_trips	Raw taxi trips
bronze.raw_locations	Raw lookup zones

Operations

- Load from Volume
- Add ingestion timestamp
- Store as Delta table

6.2 Silver Layer (Clean Layer)

Objective

Create trusted and validated datasets.

Transformations

- Remove null values
- Fix data types
- Standardize column names
- Remove duplicates
- Validate business rules

Tables

silver_trips

silver_locations

Example Cleaning Rules

- Negative fare amounts removed
- Invalid passenger counts filtered
- Timestamp normalization

6.3 Gold Layer (Business Layer)

Objective

Provide analytics-ready data.

Implementation uses **Star Schema Modeling**.

---

7. Data Modeling (Star Schema)

7.1 Fact Table — fact_trip

Each record represents one taxi trip.

Measures

- fare_amount
- trip_distance
- total_amount
- passenger_count

Foreign Keys

- pickup_location_id
- dropoff_location_id
- vendor_id
- datetime_key

7.2 Dimension Tables

dim_location

Column
location_id
borough
zone
service_area

---

dim_vendor

| vendor_id | vendor_name |

---

dim_datetime

| date_key | day | month | year | weekday |

---

dim_ratecode

| rate_code | description |

---

8. Slowly Changing Dimension (SCD)

8.1 Target Dimension

dim_location

Location attributes may change over time.

8.2 SCD Type 1

- Old value overwritten
- No history maintained

Use case:
Data correction.

8.3 SCD Type 2

History maintained.

location_id zone start_date end_date is_current

Behavior:

- Existing record closed
- New record inserted

---

9. Join Operations Practiced

Join Type	Use Case
Inner Join	Trips with vendor data

Join Type	Use Case
Left Join	Trips with location lookup
Right Join	Data validation
Full Join	Data reconciliation
Broadcast Join	Small lookup optimization
Self Join	Pickup vs dropoff comparison

---

10. End-to-End Pipeline

Step 1 — Ingestion

- Download monthly data
- Upload to Unity Volume
- Incremental ingestion using Auto Loader

Step 2 — Bronze Layer

- Raw ingestion
- Delta storage
- Metadata tracking

Step 3 — Silver Layer

- Data cleaning
- Validation
- Deduplication

Step 4 — Gold Layer

- Star schema modeling
- Fact and dimension creation
- SCD implementation

---

11. Enterprise Concepts Covered

Concept	Implemented
Delta Lake	Yes
Medallion Architecture	Yes
Unity Catalog	Yes
Volumes	Yes
Star Schema	Yes
SCD Type 1 & 2	Yes
Incremental Load	Yes

---

12. Project Outcome

Final data flow:

Raw Files (Volumes)

↓

Bronze Delta Tables

↓

Silver Clean Tables

↓

Gold Star Schema